

Script control flow

A one-liner is fine until you need decisions and repetition

If, for, and case

- An if test with brackets checks a condition, file exists, string empty
- A for loop walks a list, often a glob of files, and runs the body once per item
- A case statement matches one value against patterns and is cleaner than a long if-elif
- Quote variables, end case arms with double semicolon

Browser lab

Digital Design and Verification Platform

HomeToolsScripting

Step a tiny bash-like script: arguments, if, loops, and set -e. Lab mode — not a full shell.

Starter example: safe-args with \$1=alice → hello, alice, exit 0.

Load starter example

Challenges 0 / 23 cleared

Hello alice: With safe-args and arg alice, exit 0 and greet alice.

Load challenge setup

Show hint

Run & Check

Next

Idle

Hello alice

Missing arg - 2

set -e stops

Loop done

case build

case usage

alias ll

read bob

exit 1

exit 0

for 1..4

argc=2

true then echo

false without -e

require arg ok

need two args

case iverilog

phase wave

alias sim

fn greet

read who

early exit 2

exit 3

Presets

Load a script, set args, Run. Watch the step trace and final exit code.

safe-args

set-e-trap

loop-files

case-mode

alias-fn

read-input

exit-codes

exit-zero

for-count

nargs

true-ok

false-continue

require-arg

require-two

case-tool

case-phase

alias-sim

fn-greet2

read-echo

loop-dirs

early-exit

set-e-true

for-bits

case-empty

fn-twice

alias-chain

exit-three

script.sh

☒ honor set -e

```
#!/usr/bin/env bash
set -e
name="$1"
if [ -z "$name" ]; then
  echo "usage: $0 <name>"
  exit 2
fi
echo "hello, $name"
exit 0
```

\$1 \$2 ..

alice

stdin (for read)

bob

Run

Trace

→ set -e (errexit on)

? unknown: name="alice"

exit 127

Lab semantics

- Supported: echo, exit, set -e, if [-z "\$1"], for, case, alias, fn(), read, true/false
- With [set -e], a failing command stops the script immediately
- Exit 0 = success; non-zero = failure (scripts and Make care about this)

Scripting lab

learn_unix — scripting

Real shell practice

```
wsl - module16-scripting/examples/control_flow

# Track A - real Unix
# real shell session (Track A)

$ chmod u+x count_lines.sh

$ ./count_lines.sh
a.log: 3 lines
b.log: 5 lines
c.log: 2 lines

$ wc -l *.log
 3 a.log
 5 b.log
 2 c.log
10 total
```

Real shell — control flow script

Real shell practice — try these

```
# chmod u+x count_lines.sh — make the control-flow script executable
chmod u+x count_lines.sh

# ./count_lines.sh — for each *.log: if it is a file, print line count
./count_lines.sh

# wc -l *.log — compare: word-count lines on the same logs
wc -l *.log
```

Pitfalls to watch

- Spaces matter inside test brackets, write `[ -f "$file" ]`, not jammed tokens
- Always quote `"$file"` and `"$1"` so names with spaces survive
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, finish a few challenges after the starter
- On the real shell, run the control-flow script and try arguments or case greet next
- When you are ready